

Exercício 17 — Comparação Experimental: Fatorial

Compare as versões iterativa e recursiva do fatorial em termos de tempo, memória e facilidade.

Implementações

```
/* Iterativa */
long long fatorial_iterativo(int n) {
    long long r = 1;
    for (int i = 2; i <= n; i++) r *= i;
    return r;
}

/* Recursiva */
long long fatorial_recursivo(int n) {
    if (n <= 1) return 1;
    return n * fatorial_recursivo(n - 1);
}
```

Resultados Experimentais

N	Iterativo	Recursivo
5	120	120
10	3.628.800	3.628.800
15	1.307.674.368.000	1.307.674.368.000
20	2.432.902.008.176.640.000	2.432.902.008.176.640.000

Os resultados são idênticos. Para N pequeno (≤ 20), o tempo de execução é praticamente igual (microsegundos).

Comparação Detalhada

Critério	Iterativa	Recursiva
Tempo de execução	$O(n)$ — sem overhead de chamadas	$O(n)$ — com overhead de empilhamento
Uso de memória	$O(1)$ — constante (só resultado e i)	$O(n)$ — proporcional a n (frames na pilha)
Risco de stack overflow	Impossível	Possível para n muito grande
Facilidade de implementação	Simple — um laço for	Muito simples — segue a definição matemática
Depuração (debug)	Mais fácil — fluxo linear	Mais difícil — execução aninhada

Qual escolher em sistemas com recursos limitados?

ESCOLHA: VERSÃO ITERATIVA**Justificação:**

- Memória constante $O(1)$: crítico em microcontroladores e sistemas embarcados onde a pilha pode ter apenas alguns KB.
- Sem risco de stack overflow: mesmo para N grande, não há acumulação de frames na pilha.
- Desempenho ligeiramente superior: sem overhead de chamadas recursivas.
- Comportamento previsível: importante em sistemas de tempo real onde a latência deve ser controlada.

A versão recursiva é preferível apenas quando:
- A legibilidade é prioritária e N é garantidamente pequeno.
- O algoritmo é inerentemente recursivo (ex: Hanoi, travessia de árvores).
- O compilador suporta otimização tail-call (eliminação da recursão).